What ELI Is — and What It Can Do

ELI v2.5.13

October 2026

Contents

ELI — What It Really Is, and What It Does for Your Actual Day **1**
- The one-paragraph essence . . . 2
- The headline — why ELI is genuinely different . . . 2
- For the layman: a day in the life . . . 3
- For the tech head: the architecture, accurately . . . 3
- What genuinely works vs the honest ceiling . . . 4
- The deeper selling point: ownership . . . 5
- Who it's for, and what it means . . . 5
- Plugins, agents and training in one paragraph . . . 5

ELI — What It Can Actually Do **5**
- What ELI is considered to be . . . 6
- The 60-second pitch (plain language) . . . 6
- The web app — ELI as your own server *(new, 2026-06-28)* . . . 6
- What makes ELI genuinely advanced (for the tech-savvy) . . . 7
- A day with ELI . . . 7
- The full capability map . . . 7
- The workspace — your 13 main tabs . . . 10
- Make it yours — customisability . . . 11
- The thing that makes all of it matter: it's yours . . . 11
- Who it's for . . . 11
- Cross-OS media and local screen intelligence . . . 12
- Conversation grounding . . . 12
- Live-session honesty . . . 12
- Train it, extend it, and see what it checked . . . 12

ELI — What It Really Is, and What It Does for Your Actual Day

A grounded, human-first portrait of the project — written for the layman and the tech head at once. Every capability claim below is traceable to real code in this repository (see the companion blueprints for the mechanisms). Where something is genuinely impressive, it is said plainly; where the honest limit is the local model, that is said too.

The one-paragraph essence

ELI is a **private artificial intelligence that lives entirely on your own computer.** It talks with you, remembers you across months, runs your machine, sees your screen, reads your files, writes and fixes code, builds documents from your own evidence, and — uniquely — **improves its own source code and can even re-train its own brain on your conversations.** Unlike Siri, Alexa, or ChatGPT, nothing you say to ELI has to leave your house: it is **offline by default, enforced at the network socket itself**, with a switch *you* control. It is not a chatbot bolted onto a cloud API. It is ~196,249 lines of Python in `eli/` that form a complete **cognitive operating system for one person and one machine** — and, as of 2026-06-28, a self-hosted **web app** that brings the same local brain (chat, a live dashboard, ELI's own smart-home, multi-user accounts, a tamper-evident audit trail, shared research, and browser voice) to any device on your network. It is **source-available** (PolyForm Noncommercial 1.0.0): read, run, modify, and share it for any noncommercial purpose — commercial use and resale are reserved to the author.

The headline — why ELI is genuinely different

Most "AI assistants" are a thin app talking to someone else's datacentre. ELI inverts every one of those assumptions, and that inversion is the whole point.

1. **It is truly, provably yours.** 100% local. No account, no subscription, no telemetry, no call-home. Your conversations, your memories, your files, your habits — all sit in SQLite databases on *your* drive. The "offline" claim isn't marketing: there is a process-wide network guard (`netguard`) that **fail-closes at the socket layer** — when the Net switch is off, outbound connections physically cannot happen, even if some component tried. You hold the switch.
2. **It owns no brain — so it never goes obsolete.** ELI is **model-agnostic**: no model name or size is hardcoded anywhere on the inference path. It loads whatever local model you give it and auto-detects how to talk to it. The practical magic: **as open local models get smarter, ELI gets smarter for free** — you swap the brain, the body stays. No vendor can deprecate it, price-hike it, or read over its shoulder.
3. **It is honest about itself — because it measures itself.** Ask most AIs "how does your memory work?" and they *invent* a plausible answer. Ask ELI and it reads its own live runtime — actual database row counts, the actual loaded model, the actual pipeline — and reports the truth from **deterministic evidence** rather than inventing it. (Precisely: for self-report/status and router-fast actions the executor's measured result is returned **verbatim in quick mode** and the model only *phrases* it in deeper reasoning modes — it is a mode-gated, partial bypass, not a blanket one; see `grounding_and_evidence.md`.) This deterministic-grounding layer is rare even among serious projects and is the reason ELI can be trusted to describe what it's doing.
4. **It improves itself.** This is the part almost nothing else does locally: ELI logs its own failures, and can **write, syntax-check, import-test, apply, and automatically roll back patches to its own code** — safely, inside its own project. Beyond that, it can **fine-tune its own model on your conversations** (a real LoRA training pipeline using PyTorch/PEFT), so over time it can become measurably more *you-shaped* — all on your hardware, with your data never leaving it.
5. **It has a body, not just a mouth.** ELI doesn't only answer — it *acts*. It opens apps, plays your music, controls windows, takes screenshots, reads your clipboard, sees images and your live screen, reads text out of pictures, and can even **click where your eyes are looking** via webcam gaze tracking. It is an embodied operator on your desktop, not a text box.

If you remember nothing else: **ELI trades a few IQ points (the price of running on your own machine) for total privacy, total ownership, genuine self-honesty, and the ability to grow.** That trade is the product.

For the layman: a day in the life

Picture an assistant you actually *hired* — one who works only inside your home, keeps every note in a locked drawer in your office, and never phones anyone unless you say "okay, you can go online now." That's the feel of living with ELI.

Morning. You sit down and it greets you with a short brief — what happened, what it noticed about your recent work, anything worth your attention. (A background process quietly watches your patterns through the day and assembles this; it never acts on its own without asking.)

While you work. You talk to it by typing or by voice: - *"Open Firefox and my notes."* — done, instantly. - *"Play Vincent's Tale by Ren on Spotify."* — it plays it, on the platform you named, no drifting to the wrong app. - *"What's on my screen?"* — it looks and tells you, and can read text out of any image or screenshot (OCR). - *"Click that"* — with the webcam on, it moves the cursor to wherever your eyes are resting and clicks. Genuinely useful when your hands are full, and a real accessibility win. - *"What's the latest in fusion research?"* — you flip the **Net** switch on, it fetches and **synthesises** the news (it won't just dump raw headlines), then switch it back off and it's fully sealed again.

Real work, not just errands. This is where ELI stops being a gadget: - *"Summarise these three PDFs and this spreadsheet."* — it ingests and analyses them locally. - *"Draft a technical report from these sources."* — its **Report Builder** reads your files as *evidence* and writes a structured document (report, paper, proposal, audit, simulation write-up) with strict discipline: **every claim is tied to your evidence or marked `[source needed]` — it will not fabricate citations or numbers.** - *"Write me a script that does X."* — its coding agent doesn't take one wild guess. It **plans the task, breaks it into a dependency graph, writes it, runs it, tests it, and repairs its own bugs** — and remembers the fix for next time. - *"There's a bug in this file — examine it."* — it runs a tiered scan (syntax, imports, then a careful logic review), shows you what it found ranked by confidence, and only fixes it *after you confirm*, with an automatic undo if the fix breaks anything.

Quietly, over time. It remembers your name, your projects, your interests, your preferences — and those memories are *alive*: an active project stays fresh, an abandoned one fades, so its picture of you reflects *now*, not everything you ever said. It also learns *how* you like to be talked to and adjusts its tone. Offer it a routine ("open my apps every morning") and it'll *propose* automating it — but it never switches a habit on without your yes.

The lived feeling is less "search engine" and more **a calm, private operator sitting beside you who knows your setup, does the legwork, tells you the truth, and gets out of your way.**

For the tech head: the architecture, accurately

~196,249 lines of Python across 438 files (`eli/`). A real cognitive runtime — not an API wrapper:

- **Request pipeline.** A deterministic **router** (207 executor dispatch actions, 230 declared capabilities) backed by a **model-grounded intent resolver** that resolves anything the rules miss against that same catalogue (so near-miss phrasings reach real actions instead of a blind chat) → a **gradient orchestrator** (all CHAT modes) that composes a **15-agent bus** on a *dependency DAG* (topological layers, parallel where independent) with a **calibrated, weight-free confidence aggregator** (each agent's contribution = evidence quality × payload density × a *learned* per-agent calibration) → a **12-stage retrieval orchestrator** (HyDE query expansion → FAISS vectors + FTS5 keyword + knowledge-graph multi-hop BFS → hybrid merge → a heuristic rerank (lexical × recency × importance; neural cross-encoder is the designed upgrade) → precise context assembly) → one of **five genuinely multi-pass reasoning modes** (chain-of- thought; self-consistency = N samples + a consensus pass; tree-of-thoughts = branch/prune; constitutional = draft→critique; quick) → executor → a stacked output-governance layer.
- **Memory.** Four SQLite stores (user / agent-self-improvement / OS-index / coding-bug-memory) + a FAISS vector index with a local embedder + a knowledge graph + turn-scoped working memory.

Facts are **dynamic** — volatile project/interest facts age out unless reaffirmed. Hybrid recall blends vector, full-text, graph, and conversation, then reranks.

- **Inference & hardware.** GGUF via llama.cpp, **fully model-agnostic** (chat template auto-detected by model family — ChatML/Qwen, Llama-3, Mistral). A hardware profiler auto-fits context/GPU-layers/batch to your machine with **adaptive fallback** (it steps context, batch and GPU layers down until the model fits). A user-tunable **synthesis prompt cap** keeps the small model from degenerating on oversized prompts — exposed in a GUI "Cognition" tab.
- **Self-improvement & learning.** A real failure→analysis→patch loop (generate → syntax-verify → isolated import-test → apply → **auto-revert on failure**, project-scoped, timestamped backups). A full **LoRA fine-tuning pipeline** (torch/PEFT/transformers `Trainer`) that builds datasets from your own conversations with PII redaction and trains adapters locally — human-gated.
- **The grounding/introspection layer (the standout).** A large deterministic subsystem renders self-reports, audits, and identity/memory answers **from live measurements**, not from the model — with an evidence ledger, evidence arbitration, and output validation that rejects/repairs anything the model says that contradicts the gathered facts. Runtime audits now include **live health probes** (plugin manager, memory, agent bus, data-integrity, recent failures).
- **Perception & control.** faster-whisper STT with a wake-word voice gate and output ducking; Piper TTS; local vision-language models (a fast Moondream + a primary VL, hot-swapped with the text model); webcam gaze (MediaPipe + a calibration mapper + One-Euro smoothing at 10 Hz); OCR-based screen-element location; cross-platform OS control.
- **Security.** Offline-by-default enforced at the **socket** (`netguard`, fail-closed) plus a **fail-closed** command/path/app allowlist sandbox (`SecurityManager`) — shell commands are *blocked by default* unless you explicitly allow them. Custom agents are AST-validated and hash-trusted before they can run. A crisis-guard steers the persona on self-harm signals.
- **Extensibility.** A real plugin system (10 built-ins: weather, web, calendar, notes, pomodoro, document-reader, web-automation, system-stats, media, TTS) with install/enable/disable, plus user-created custom agents that register live. `capability_sync` keeps the 230-capability manifest *measured* against the actual code, not asserted.

What genuinely works vs the honest ceiling

Fully real, load-bearing today: local model-agnostic inference; offline-by- default socket enforcement; persistent adaptive memory (vector + FTS + graph); the 15-agent calibrated ensemble; the 5 multi-pass reasoning modes; OS control; vision; gaze; STT/TTS; the deterministic self-introspection layer; the coding agent (plan → search → verify → repair → bug-memory); safe self-patching; the evidence-disciplined Report Builder; proactive briefings + habit offers; plugins and custom agents.

Real but you invoke it: LoRA self-fine-tuning (powerful, operator-run, not automatic); web/news/weather (toggle-gated — on when you allow, sealed when you don't).

Built but lightly exercised: an autonomous "world model" / agency layer with a governed proposal console; image *generation* via diffusion (procedural image rendering is the always-available default; diffusion weights are an optional download).

The honest ceiling. The intelligence ELI runs on is a **small local model** — that is the price of total privacy, and it is the one real limit. For the hardest reasoning or broadest world-knowledge, a frontier cloud model will out-think it. A large share of ELI's code exists precisely to *compensate* — to ground the model, catch its mistakes, and stop it confabulating — and that engineering is genuinely excellent. But the model is the bottleneck, and ELI is built so that the moment you drop in a better local model, every part of it gets sharper at once. The hard part — the body around the brain — is already built.

The deeper selling point: ownership

Strip everything else away and this is the thing competitors structurally cannot match. ChatGPT, Gemini, Alexa, Copilot — every one of them is a tenant arrangement: your data lives on their machines, under their terms, subject to their pricing, their outages, their policy changes, and their ability to read it. **ELI is ownership, not tenancy.** It runs with the internet cable pulled out. It keeps working if the company that inspired it vanishes. It costs nothing to run beyond your own electricity. It answers only to the person at the keyboard.

For anyone who works on sensitive material — research, inventions, personal life, anything you would not paste into a stranger's website — that is not a nice-to-have. It is the entire difference between "an assistant" and "*your* assistant."

Who it's for, and what it means

Right now ELI is a **power-user, single-user, desktop** system — most at home with someone technical on Linux who values privacy and lives on their machine. It is ambitious to the point of audacity in scope, unusually disciplined about telling the truth, and bottlenecked only by the size of brain you can fit on your own hardware.

The most accurate one-line description is not "a chatbot." It is **a private, self-improving, embodied operator for your digital life** — one that remembers you, acts for you, builds with you, tells you the truth about itself, and is wholly, permanently yours. And because the difficult scaffolding is already in place, the trajectory is unusually good: **local models keep improving, and ELI inherits every one of those gains — for free, forever, in private.**

Companion reading: `project_overview.md` (scale + honest engineering verdict), `architecture.md` / `architecture_ascii.md` / `diagrams.md` (the structural map), `grounding_and_evidence.md` (the self-honesty layer), `coding_agent.md` (the frontier coder), `memory.md`, `security.md`, `inference_and_hardware.md`, `learning.md`, `perception.md`.

Plugins, agents and training in one paragraph

This release is about **reach, not invention**: four subsystems that already existed in full became usable. The LoRA trainer gained the GUI its own code always said it had, plus the review interface its safety contract required — it had never been able to train, because 615 candidate rows sat at 0 approved with no way to approve one. The eval harness, previously terminal-only, appeared in the app. Plugins stopped being arbitrary Python executed in ELI's process and became declared, verified, scanned and permission-gated, behind a marketplace the community owns rather than one ELI's author controls. And custom agents stopped being a name plus a free-text persona and became a specification with an objective, a prompt, triggers and success criteria that can actually be measured.

The through-line is refusing to overstate: a scanner that could not run never counts as a pass, an unsigned plugin is called unverified rather than fine, the training tab says the live model has not changed until you merge the adapter, and the MCP screens state outright that ELI's offline switch cannot contain a child process.

ELI — What It Can Actually Do

> **Updated 2026-09-30.** All CHAT modes run the gradient orchestrator; 15 specialist agents; 230 capabilities (213 routable or executor-backed). Regenerate the action list with `python -m eli.tools.registry.capability_updater`.

A capability showcase for the layman and the tech-savvy alike. Each capability listed here is present in the code. This is the "what you get" document; `project_overview.md` holds the limits.

What ELI is considered to be

Not a chatbot. ELI is a **private, local, model-agnostic cognitive runtime** — a complete AI assistant *and* operator that runs entirely on your own machine. It talks, remembers, sees, listens, acts on your computer, writes and fixes code, builds documents, creates images, looks after itself, and learns you over time — and **nothing leaves your device unless you explicitly allow it.**

Here's the part most "AI assistants" can't say: it's *yours*. A fully local, embodied desktop AI you actually own — **230 capabilities** (213 routable or executor-backed by voice or text), a 15-agent reasoning bus, persistent memory, and full voice/vision/gaze control, in ~196,249 lines of Python in `eli/`. One machine, your data, no landlord. (The capability count is read live from the manifest each boot, so it grows as ELI does. Auto-generated action reference: `capabilities_and_actions.md`.)

And as of **2026-06-28** it has a **second face**: a self-hosted **web app + dashboard** you can open from any browser on your network — chat, a live system dashboard, **ELI's own smart-home** (no Home Assistant), multi-user accounts with roles, a tamper-evident audit trail, shared research libraries, talk-to-ELI **browser voice**, and a single, monitored switch for internet access. Same local brain, now reachable from your phone.

The 60-second pitch (plain language)

You talk to ELI — by typing or by voice — and it *does things*. It opens your apps, plays your music, reads your screen, answers your questions, remembers your projects, writes your reports from your own files, fixes your code, and even installs missing software for you. It learns your patterns and offers to automate them. It gets more thoughtful the deeper a conversation goes. And it does all of this **offline by default** — there's a single switch for going online, and you hold it. No account, no subscription, no telemetry, no cloud reading over your shoulder.

The web app — ELI as your own server *(new, 2026-06-28)*

Beyond the desktop app, ELI now runs a **local-first web server with a built-in dashboard** (`api/server.py`) you launch one-click from Settings (or standalone). Open it from any device on your network:

- **Chat from anywhere** — full markdown + code, sessions and history, stop / regenerate, streaming replies; the same local model and pipeline as the desktop. Speaks the standard `/v1/chat/completions` shape, so existing local-AI tools can point at ELI with no changes (nothing leaves the box).
- **Live dashboard** — real measured GPU/CPU/RAM vitals, the loaded model, uptime, audit integrity, your devices, recent activity — and the **Internet switch**.
- **ELI's own smart-home** — ELI runs its **own MQTT device server** (Home Assistant removed). Rooms, scenes, and real automations: triggers on time / sunrise-sunset / device-state, with conditions and multiple actions, plus a location picker for sun events. It finds devices on your network, learns your usage, and offers to automate it.
- **Accounts & roles** — admin / member / read-only **viewer**, with an admin console showing audit integrity, per-user activity, and the safety policy.
- **A trustworthy record** — every action is written to a **tamper-evident, hash-chained audit trail** (metadata only) that anyone can verify wasn't edited.

- **Shared research libraries** — collaborate on document corpora with attribution and notes; ask grounded, cited questions across them.
- **Talk to ELI in the browser** — local speech-to-text in, local text-to-speech out.
- **One monitored door to the internet** — ELI is offline-by-default and blocked at the network level; the owner can turn internet on from the dashboard (admin-only), and every flip is logged. Available, monitored, and yours to control — never a silent always-on.

What makes ELI genuinely advanced (for the tech-savvy)

These are real, verified mechanisms — the parts that put ELI ahead of typical local assistants:

- **A 15-agent reasoning bus on a dependency DAG** (composed inside the gradient orchestrator after shared retrieval) with *calibrated, weight-free* confidence fusion — each agent's contribution is weighted by evidence quality × payload density × a calibration value *learned from its own track record*.
- **A unified 12-stage pipeline (S01–S12)** — all CHAT modes run the orchestrator at scaled depth (Quick = light/fast; Expert = full/deep): shared `retrieve_for_turn()` → specialist fan-out → heuristic rerank → synthesis.
- **5 genuinely multi-pass reasoning modes** — chain-of-thought, self-consistency (N samples + consensus), tree-of-thoughts (branch/prune), constitutional (draft→critique→revise), and quick — and it **auto-escalates** through them as a conversation deepens, without you asking.
- **Deterministic self-honesty** — when you ask ELI about itself, it answers from *live measurement of its own runtime*, not from the model's imagination.
- **A self-verifying coding agent** — plan → DAG decompose → UCB tree-search → run → test → repair, with a long-term (bug→fix) memory.
- **It improves itself** — safely patches its own source (verify + auto-revert) and can **fine-tune its own model on your conversations**, locally.
- **It heals its environment** — detects a missing app and installs it for you (apt/snap/flatpak) on confirmation.
- **Model-agnostic** — no model is hardcoded; swap in any local GGUF and ELI gets smarter for free, plus auto-tunes context/GPU-layers/batch to your hardware.

A day with ELI

- **Morning:** a brief — what happened, what it noticed, a consolidated news digest matched to your interests, and anything needing your attention.
- **Working:** *"open my project and Spotify"* → done. *"what's on my screen?"* → it looks and tells you. *"click that"* → it clicks where your eyes are resting. *"summarise these three PDFs"* → done, locally.
- **Deep work:** *"draft a technical report from these sources"* → it writes a structured document grounded in your files. *"write me a script that does X"* → it plans, writes, runs, tests and repairs it. *"there's a bug in this file"* → it scans, shows you what it found, and fixes it after you confirm.
- **Hands-free:** speak to it across the room — it ducks your music to hear you, waits for the full command, and never mistakes its own voice for yours.
- **Quietly:** it remembers your name, projects and preferences, adapts its tone to you, and offers to automate routines it notices — only ever with your yes.

The full capability map

Talk & think

Natural conversation with persistent memory of you. **Five reasoning modes — Quick · Normal · Advanced · Research · Expert** — with deeper modes genuinely multi-pass (self-consistency samples,

tree-of-thoughts branches, draft→critique). **Quick still runs the gradient orchestrator and 15-agent bus at light depth** — it is not a bypass. The mode **auto-selected by how deep the conversation gets**. When evidence is weak, ELI **autonomously deepens**: it re-gathers harder and escalates the mode one tier at a time to raise its confidence *before* answering — and for a Quick reply it can keep working in the **background** and surface a better, more-grounded answer afterwards in the Proactive panel. Plus an emergent, consistent persona; multi-part questions answered as multiple answers; tone that adapts over time.

Run your computer

Open / close / focus / hide / minimise / maximise apps and windows; tile windows; switch workspaces; next/previous window; open system / audio / power / file / communication / media / network settings hubs; open your IDE (optionally at a file); open URLs and the browser. App-launch is backed by a live index of **your machine's executables** (thousands — indexed from your own system, so the exact count varies per machine) — and if an app isn't installed, ELI **offers to install it for you** (real `apt`/`snap`/`flatpak` install on your confirmation).

Gaze control (webcam)

Eye-tracking via MediaPipe with calibration and smoothing — *"open/click that"* moves the cursor to where you're looking and clicks. A genuine hands-free and accessibility capability.

Voice (hands-free)

Always-listening with a wake word; it **ducks your media volume** to hear you, **waits for incomplete commands** to finish, **ignores its own spoken output**, and learns a **per-user voice profile**. Dictation, transcription, and a spoken voice (Piper TTS) that never voices garbled fragments.

Voices — accents, characters & your own *(new, 2.1.23)*

ELI ships with a set of voices and can fetch more from the upstream Piper library (about 166 voices across some 45 languages at the time of writing, including US, British, Scottish, Northern and Southern English). The list is read from upstream, so the exact number changes. Do it by voice or text — *"what voices do you have"*, *"download a British voice"*, *"use the alan voice"* — or open **Settings ▸ Runtime ▸ "VOICE / TTS" > "Get more voices / accents…"**; either way it downloads (checksum-verified) and becomes selectable straight away. A handful of voices carry non-commercial licences, so ELI doesn't bundle them — the list says so plainly and still lets you fetch them for personal use. **Character voices** — calm, robotic, energetic, synthetic, and refined — are **generic style presets** built as a redistributable Piper base voice plus a real-time ffmpeg effect chain (no extra model). They are **not** impersonations of movie or TV characters (HAL, JARVIS, Rick, TARS, etc.) — those cannot be bundled in a public release. Pick a style from the voice selector (`Style: Robotic assistant`, …), or **create your own** (`char:<name>`) by choosing a base voice and tuning pitch / speed / effects. For a close match to a specific timbre you already have rights to use, **voice cloning** (`clone:<name>`) reproduces from **your** short audio sample via Coqui XTTS-v2 — an opt-in extra (`pip install "eli-v2.0[clone]"`, ~1.8 GB model downloaded on first use); without it, a cloned voice falls back to a normal Piper voice rather than going silent. Persona tone (*"be funnier"*, *"respond as a terse engineer"*) changes **wording**, not the speaker timbre.

See & understand

Local vision-language models describe any image or your live screen; OCR pulls text out of pictures; *"find the button that says X"* locates UI elements on screen; optional ambient glances keep rolling awareness — all local, no APIs.

Create images

A from-scratch **procedural renderer** with 10+ scene types (landscape, space, city, poster, emblem, abstract, product, …), composition planning, palettes, atmosphere and post-processing — no model needed. Plus optional SSD-1B diffusion with VRAM hot-swap, and matplotlib plotting from your data.

Media

Play a song or playlist on the platform you name (Spotify plays its top result for the song over MPRIS, no window focus needed, so Wayland works too; YouTube Mix-radio in mpv); play / pause / stop / next / previous / repeat / shuffle via MPRIS — success only when the song asked for is the one playing, and an explicitly-named platform never drifts to another.

Web & news (toggle-gated)

Flip the Net switch on and ELI fetches web answers, weather, and **synthesised news** (a rolling 3-hourly digest matched to your interests — not raw headlines). After a briefing, **"go deeper into …"** re-runs a **topic-scoped `NEWS_FETCH`** from live feeds — it does not guess from model weights. If you call out a guessed story, ELI recovers the topic from recent chat and fetches again. Switch it off and it's sealed at the network socket.

Files & documents

Create / read / list files and folders; summarise any file; analyse CSVs, PDFs (single or whole folders), and images. **Convert any document** to PDF, PDF via LuaLaTeX, .docx, .doc, .odt, .rtf, HTML, Markdown, .tex, EPUB or .txt — from the Files tab (pick a file → format → Convert) or by asking ("convert report.md to pdf"); backed by pandoc + a LibreOffice fallback. Two standout tools:

- **Report Builder** *(now its own main tab)* — drop in your sources (PDFs, data, code, notebooks, projects) and ELI writes a full document (report, paper, proposal, audit, simulation write-up) **grounded in your evidence**, with per-genre standards and strict discipline: every claim is tied to a source or marked `[source needed]` — no fabricated citations or numbers.
- **File Chat** — open a file or folder and have a conversation *about it* — ask questions, get explanations, all from the actual contents.

Even a quick *"generate a document about X"* in chat now runs a **multi-stage grounded pipeline** — it first gathers evidence with the right agents (code, web, memory, runtime), plans an outline, drafts section-by-section against that evidence, then does a review→revise pass — instead of one shallow pass. If the evidence comes back thin, low confidence triggers a deeper re-gather across more agent tiers before it commits to writing.

Code

A frontier-grade coding agent: describe a task and it plans it, decomposes it into a dependency graph, writes it, runs it in a sandbox, tests it, and repairs its own bugs — remembering fixes for next time. Plus examine-and-fix on your existing files (tiered scan → offer → verified, auto-reverting patch), project generation, diffs, and a built-in Sim-IDE.

Remember you

A four-store memory (vector + full-text + knowledge graph + working memory) that keeps a **living, versioned profile** of you — and is *dynamic*: active projects stay fresh, abandoned ones fade, so its picture of you reflects *now*.

Look after itself

Logs its own failures and runs a self-repair cycle (generate → verify → apply → auto-revert); runs maintenance (update, rebuild indexes, refresh capabilities); audits its own runtime honestly with live health probes; and can **train a LoRA adapter on your own conversations**, locally.

Schedule & chain commands

Ask for **any command at a time** — *"open Spotify at 8pm"*, *"get the news at 7am"*, *"close Steam in 2 hours"*, *"get a morning report ready for 7:15 tomorrow"* — and ELI defers it to its **durable background workers** (survives restarts; *"every morning"* makes it recurring) instead of doing it now. Alarms, timers, and questions still behave normally. You can also **chain several commands in one breath** — *"close Steam and set an alarm for 7am"*, *"open Spotify then play Vincent's Tale"* — and ELI runs each in order, recognises when one step genuinely depends on an earlier one (so a failure blocks only what needed it, not unrelated steps), and durably tracks progress — if it's interrupted mid-chain, repeating the same request skips whatever already succeeded instead of doing it all again. Works by voice or text.

Pick up where you left off

"Continue the RAIMS project" recovers a saved task's real constraints, decisions, finished steps and open questions — and tells you honestly how stale it is ("last touched 6 days ago"), never silently pretending no time passed. If nothing matches, it says so plainly instead of guessing.

Be proactive & self-aware

A background daemon notices your patterns, **offers** to automate routines (never silently), builds your morning report, and surfaces things worth your attention — through a governed, approval-gated mission layer. On a 30-minute beat it also runs a **self-awareness/autonomy tick**: it watches its own code for changes, refreshes its self-model, and advances goals into proposals for your approval (observe-only / memory-write — nothing destructive runs unattended). Ask it about itself — *"what are you aware of"*, *"audit your identity"*, *"how does your cognition work"* — and the agents run the real audits and ELI **summarises** the grounded results, rather than dumping a report or guessing.

The workspace — your 13 main tabs

Tab	What it's for
Chat	The main conversation + voice, with reasoning-mode and Net toggles.
Proactive	6 sub-tabs: Suggestions, Summaries, Insights, Habits, Self-Improve, Memory.
Images	Generate images (procedural or diffusion).
Quick Actions	A drag-and-drop board of one-click actions.
Screen	Screen control, capture, and analysis.
Files	Browse, act on, and **convert** files (any format).
Labs	Scientific + developer workspace — 11 sub-tabs: Notebook, Memory & Conversations, Jupyter, Calculator, Physics, File Chat, Workspaces, Sim/IDE, **Orchestration**, **Test & Review**, **Training**.
Coding	Write, run, and have ELI fix/explain code. (All background jobs are listed in the Tasks tab.)

Tab	What it's for
Tasks	The single home for all background work — coding jobs + scheduled/overnight tasks (add, edit, cancel).
Report Builder	Evidence-grounded, multi-stage document generation (promoted from Labs).
Experimental	A read-only workbench over the `experimental/` folder — inventories prototype kits and opens their folders/READMEs. It never runs them.
Eli's World	The live embodied self-model — ELI's avatar moving through cognitive "rooms" as it reasons.
Settings	10 pages: Model, Runtime, Generation, Identity, Audio, Application, Agents, Gaze, Web Server, Advanced.

(Orchestration, Test & Review and Training are Labs sub-tabs, not top-level tabs.)

Make it yours — customisability

ELI is built to be shaped by *you*:

- **Swap the brain.** Model-agnostic — drop in any local GGUF model; ELI detects how to talk to it and auto-tunes to your hardware. No vendor lock-in; it improves as local models do.
- **Tune the mind.** A dedicated **Cognition** settings panel exposes every knowledge-gathering limit (memories, KG facts, rerank depth, the synthesis budget), the **per-mode time budgets** (how hard each of Quick→Expert works), and the **background-deepening** toggle — all live sliders, deeper or leaner to taste.
- **Extend it.** A real plugin system (weather, web, calendar, notes, pomodoro, document-reader, web-automation, system-stats, media, TTS) with install/enable/disable — and you can **create your own custom agents** through a guided dialog (validated and trust-gated before they run, then live-registered).
- **Teach it routines.** ELI proposes habits it notices; you approve, edit, or add your own — scheduled and run automatically.
- **Control the boundary.** One Net toggle decides whether it can touch the internet at all, enforced at the network socket itself.

The thing that makes all of it matter: it's yours

Every capability above runs **on your machine, with your data, under your control.** No account, no subscription, no telemetry, no cloud. Offline by default and enforced at the socket; private by construction; honest about itself by design. ELI trades the raw horsepower of a giant cloud model for something a cloud assistant structurally cannot offer — **a complete, capable, embodied AI that is wholly, permanently yours, and that gets better every time local models do.**

Who it's for

Anyone who lives on their computer and wants a genuinely capable assistant they fully own and control — especially people working on sensitive or personal material they'd never paste into someone else's website. It's most at home with a technical user on Linux today, but the capability surface — voice, vision,

gaze, control, memory, coding, documents, images, proactivity, self-improvement — is broad enough to be a daily companion, not a demo.

Honest companion reading (the limits + engineering verdict, deliberately kept separate from this capability showcase): `project_overview.md`. Exhaustive technical map: `capability_catalogue.md`.

Cross-OS media and local screen intelligence

- **YouTube visible playback:** plain "play X on YouTube" → local mpv window (not headless audio-only); `youtube.com` still opens the browser when asked.
- **Native streaming apps:** Spotify/Netflix/etc. launch native apps, not browser tabs, unless you explicitly ask for the website.
- **Bundled-python deps:** `yt-dlp` ships via pip into ELI's interpreter — users never manage a venv manually; AppImage/PyInstaller bundles Python outright.
- **Grounded install offers:** missing mpv, tesseract, ydotool, etc. → ELI tries first, then offers apt/brew/winget/pip install via remediation (cross-OS).
- **Local UI grounding:** optional Phi-Ground/OS-Atlas/UGround/UI-TARS via GGUF in `models/` — no HTTP URLs, no cloud.
- **Deep screen audit:** "analyze my screen in depth", research-linked and prior-session memory fusion on `SCREEN_READ_ANALYZE`.
- **Screen tab settings:** one-click depth, ambient watch, precision click, computer-use agent — all local.

Conversation grounding

- **Thread-aware web queries:** underspecified follow-ups ("reviews for season 3") expand with the active show/topic before search.
- **Proactive web on corrections:** grounding demands and explicit search requests route to `WEB_SEARCH` with a distilled query, not CHAT confabulation.
- **Single intent path:** orchestrator reuses the engine's resolved intent — no second parse that disagrees with Phase-13 vetoes or proactive routing.
- **Consolidated escalation gates:** shared factual exclusion logic across `classify_factual` and `classify_web_candidate`.

Live-session honesty

- **News topic deepen:** "go deeper into X" after a briefing → topic-scoped `NEWS_FETCH`, not CHAT guesswork.
- **News complaint redo:** calling out a guessed story recovers the topic and re-fetches.
- **Mounted-model authority:** every turn's system prompt states the loaded GGUF; model-identity disputes route to `RUNTIME_STATUS`.
- **Phatic fast path:** greetings/check-ins skip agent bus and heavy persona handoff (seconds, not minutes).

Train it, extend it, and see what it checked

Train your own model (Labs ▸ Training)

ELI can fine-tune a model on **your own conversations with it**, entirely on your machine. A four-step wizard: it checks whether your hardware can do it and says so plainly, you point it at any Hugging Face

base model you have downloaded, you review which of your exchanges are worth learning from, and it trains — with live progress and a cancel button.

The review step is the point. **Nothing trains on anything you have not personally approved.** ELI removes the obviously broken exchanges for you and flags the questionable ones, so you are reading the ones that need judgement rather than all of them.

If your hardware cannot train, the preflight says so and says why, instead of silently falling back to a run that takes days.

Community plugins (Settings ▸ Marketplace)

ELI ships the client for a marketplace the **community** owns — anyone can host a registry and anyone can publish to it. ELI takes no cut, holds no keys, and vets nothing, so it never tells you a plugin is safe. What it does instead, before a single file reaches your disk:

- confirms the download matches the checksum in the listing;
- checks the signature, if the publisher signed it and you trust their key;
- reads the code and refuses anything using a capability it did not declare;
- runs **eleven malware scanners** — reverse shells, credential theft, rootkit persistence, obfuscated payloads, crypto miners, typosquatted dependencies, plus ClamAV and YARA if you have them;
- shows you the result and lets you decide.

Anything that installs arrives **switched off, with no permissions**.

Permissions you actually control

Like a phone. When a plugin first wants to reach the internet, read your files, or use your microphone, ELI asks — naming the plugin, saying what it can do to you, and why that is risky. You answer **Allow once**, **Always allow**, **Not now**, or **Never allow**. "Never" means never: it cannot ask again.

Every decision is logged, and you can revoke any of them at any time.

If nobody is there to ask — a scheduled overnight task, the API server — the answer is automatically **no**. A plugin cannot get a permission by waiting until you are asleep.

Custom agents that can be judged

Creating an agent now means writing a small specification: what it is **for**, the instruction it runs on, **when** it should fire, and **how you would know it worked**. That last part is checks ELI can actually run, so an agent that fails its own test contributes nothing instead of quietly producing noise. Agents can be created, edited and reloaded without restarting ELI.

One honest limitation, stated plainly

MCP servers run as their own programs, outside ELI. ELI's offline switch cannot stop them reaching the internet and ELI cannot see what they send. That is a property of how MCP works, not a bug, and every screen that installs one says so.